SmartChain: A Dynamic and Self-Adaptive Sharding Framework for IoT Blockchain

Ting Cai, *Member, IEEE*, Wuhui Chen, *Member, IEEE*, Jianting Zhang, and Zibin Zheng, *Fellow, IEEE*

***Abstract*—Sharding technologies allow the Internet of Things (IoT) to deploy blockchains in large-scale applications with good scalability. However, conventional sharding strategies in IoT blockchain are highly restricted because most IoT devices are dynamic and heterogeneous. They fail to partition and reconfigure shards with a fine-balanced tradeoff between throughput and security. Therefore, we propose SmartChain, which is a dynamic and self-adaptive sharding framework devised for making sharding decisions on the IoT blockchain featured with dynamics and heterogeneity. Specifically, we elaborate on how SmartChain performs reconfiguration and provide a quantitative analysis of shard performance. We then formulate the long-term tradeoff of throughput and security as a Markov decision process. Considering the nature of time-varying devices (e.g., amount of computing power, location), we develop a Transferable Proximal Policy Optimization (PPO) with Demonstrations algorithm, namely TPPOD, to help quickly reconfigure shards when the environment changes. Thus, based on current state, SmartChain can adaptively and dynamically select shard number, partition structure, and primary selection mode. Evaluations show that SmartChain enables high throughput and low risk of security, and reduces 70% of the training time averaged over baselines. Our implementation of TPPOD is 8.3 times of average system reward compared with the PPO-based sharding strategy with uniform sampling.**

***Index Terms*—Blockchain, deep reinforcement learning, IoT devices, sharding.**

I. Introduction

THE rapid growth of Internet of Things (IoT) has led to major challenges in decentralization, mutually-distrusted devices and systems, privacy and security vulnerabilities.

Manuscript received 6 December 2022; revised 26 December 2023; accepted 15 February 2024. Date of publication 18 March 2024; date of current version 10 April 2024. This work was supported in part by National Key Research and Development Plan under Grant 2021YFB2700302, National Natural Science Foundation of China under Grant 62302154, and Grant 62172453, National Natural Science Foundation of Guangdong province under Grant 2022A1515010154, Major Key Project of PCL under Grant PCL2023AS7-1, Pearl River Talent Recruitment Program under Grant 2019QN01X130, Doctoral Scientific Research Foundation of Hubei University of Technology under Grant XJ2022006701, and Key Research and Development Program of Hubei Province under Grant 2023BEB024. *(Corresponding author: Wuhui Chen.)*

Ting Cai is with Sun Yat-Sen University, Guangzhou, Guangdong 510275, China, and also with the Hubei University of Technology, Wuhan, Hubei 430068, China (e-mail: caiting@hbut.edu.cn).

Wuhui Chen is with Sun Yat-Sen University, Guangzhou, Guangdong 510275, China, and also with Pengcheng Laboratory, Shenzhen, Guangdong 518066, China (e-mail: chenwuh@mail.sysu.edu.cn).

Jianting Zhang is with the Department of Computer Science at Purdue University, West Lafayette, IN 47906 USA (e-mail: zhan4674@purdue.edu).

Zibin Zheng is with Sun Yat-Sen University, Guangzhou, Guangdong 510275, China (e-mail: zhzibin@mail.sysu.edu.cn).

Digital Object Identifier 10.1109/TSC.2024.3376242

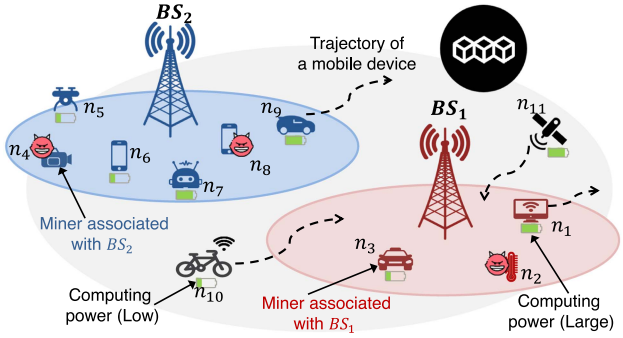


Fig. 1. Illustrations of the IoT blockchain with dynamics and heterogeneity. Here the heterogeneity is labeled by varied amount of computing power.

Blockchain technology brings promising to address the above challenges by enabling secure and reliable transactions via building blocks. In this context, a new paradigm of integrating blockchain with IoT, named IoT blockchain [1], is being investigated. Though the IoT blockchain has great potential to release, there exist scalability issues. For example, Ethereum has the capability of 14 Transactions Per Second (TPS), which capabilities are far below the peak of 59,000 TPS required by the IoT, e.g., a midget smart factory [2]. Therefore, how to improve the scalability is an issue of great significance for applying IoT blockchain in large-scale real-world applications.

Recently, sharding [3] emerges as a technique used in blockchain to enhance scalability by dividing the blockchain nodes into multiple smaller partitions, called committees or shards, which can process transactions in parallel, and thus TPS can be enhanced. In this context, several sharding systems have been proposed, including Elastico [4], Rapidchain [5], OmniLedger [6], Monoxide [7], Optchain [8], and Polyshard [9]. All of these sharding systems assume that all blockchain nodes are stable and homogeneous in a peer-to-peer network; thus, they adopt random and static sharding strategies, i.e., randomly allocating the blockchain nodes to different shards with fixed sharding settings. However, such sharding strategies are not applicable for the IoT blockchain system with dynamics and heterogeneity. We discuss the limitations of previous sharding strategies using a motivation example as shown in Fig. 1.

Random sharding cannot reach a long-term balance between throughput and security, see Fig. 2(a) and (b). Fig. 2(a) gives a result of randomly assigning malicious IoT devices (n_2, n_4, and n_8), i.e., high long-term throughput, but low long-term security. Devices of the same color connect to the same base

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

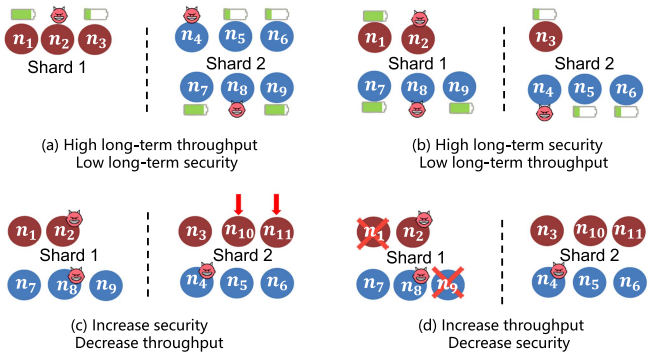


Fig. 2. Issues of random and static sharding strategies, e.g., (a), (b) show the result of two random sharding examples; (c), (d) display two results of static reconfiguration.

station, and rely on this base station for communication. Thus, partitioning IoT devices connected to the same base station into one shard can bring high throughput by reducing communication latency, but leading to low long-term security since adversaries can invalidate the shard by attacking the only base station on which all intra-shard devices rely for communication. Fig. 2(b) shows another result of random sharding, i.e., high long-term security, while low long-term throughput. Assigning IoT devices connected to different base stations to one shard can improve security due to improved robustness. Whereas, shard 2 with low average computing power will be a drag on the overall throughput. Also, the long-distance communication of IoT devices within each shard affects the throughput. Therefore, previous random sharding strategy only obtains a probabilistic safety guarantee at the beginning of shards formation. It compromises the long-term balance between throughput and security when sharding for heterogeneous IoT blockchains.

Static sharding faces a dilemma that how to balance throughput and security during reconfiguration, see Fig. 2(c) and (d). Since adversaries will adaptively attack on IoT devices and base stations [4], [5], [6], [10], shards need to be shuffled, called reconfiguration [3]. However, shuffling shards in a static manner, e.g., fixed-interval reconfiguration, fixed shards number, is not suitable for dynamic IoT blockchains. Fig. 2(c) describes the shards are reconfigured in a fixed short interval. High-frequency shuffling increases the security of shards, however, new joining IoT devices, like n_{10} and n_{11}, may have not enough time to perform state synchronization or consensus in shard 2, thus compromising the throughput. Fig. 2(d) shows the impact of a fixed large reconfigured interval. Low-frequency shuffling enhances the throughput but decreases security, providing adversaries with a longer time to perform adaptive attacks. Moreover, the dynamics of IoT devices exacerbates security risks because a reduction in the size of shard is easy to occur, especially over a long period of time. In Fig. 2(d), as results of leaving like n_1 and n_9, shard 1 turns invalid because there are more malicious devices than honest devices.

Therefore, a critical issue is to partition shards in a new kind of adaptive and dynamic way, such that the trade-off between throughput and security can be achieved in a long-term balance, under the scenario of dynamic and heterogeneous IoT blockchain. We formulate sharding decisions for the long-term tradeoff as as a Markov Decision Process (MDP) and use Deep Reinforcement Learning (DRL) to find the optimal sharding strategy in time-varying and stochastic environments.

Challenges: IoT features increase the difficulty of sharding, e.g., dynamic IoT devices could leave or rejoin shards, or the amount of computing power on devices might vary over time, so that some shards may be taken over by adversaries and security guarantees of the blockchain can be broken [3]. Reconfiguration should be emphasized and how to avoid the risk of state loss or synchronization conflicts during reconfiguration remains a challenge. Nonetheless, we still have not yet found any available works to consider the reconfiguration of shards while quantifying the phase in their model, especially for the IoT network featured with dynamics and heterogeneity.

On the other hand, the DRL algorithm typically requires a huge amount of data before reaching reasonable performance. This will severely limit the applicability of our proposed sharding strategy in IoT blockchain. Moreover, IoT devices are featured with dynamic mobility, thus the traveling of blockchain nodes from current environments to new environments will occur periodically. Meanwhile, dynamic adjustments on shards during reconfiguration in the network of IoT blockchains are expected to be time-limited because most widely-adopted IoT applications are required to be done in real time. Therefore, a novel DRL algorithm that can reduce reconfiguration latency then quickly adjust shards to a new environment is in urgent need.

Contributions: Facing the above challenges, we present SmartChain, a dynamic and self-adaptive sharding framework. It involves a shard reconfiguration mechanism, which can adaptively make shuffling decisions to perform shards reconfiguration according to the dynamic changes of IoT environments; meanwhile, guaranteeing that ledgers of each shard are completely kept and correctly synchronized. In addition, we introduce a Transfer Learning (TL) method [11] to develop a Transferable Proximal Policy Optimization (PPO) with Demonstrations algorithm, namely TPPOD, aiming to help IoT devices accelerate the training process in new environments by leveraging learned experiences. The contributions are highlighted as follows.

- SmartChain partitions shard and performs reconfiguration using the DRL. Thus, by adaptively and dynamically adjusting shard number, partition structure and primary selection mode, the tradeoffs of throughput and security can be achieved a long-term balance among all shards.
- To quickly make dynamic reconfiguration on the heterogeneous IoT devices, we leverage transfer learning to enhance learning capacities. By combining with a prioritized-experience replay technique, SmartChain can utilize small sets of demonstration data to massively accelerate the training process of devices.
- We conduct shard-driven simulations that demonstrate SmartChain can improve both throughput and security, reduce 70% of training time averaged over baselines, and

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

outperform the PPO-based sharding strategy with uniform sampling in terms of 8.3 times the average system reward.

The remainder of this paper is outlined as follows. Section II describes the related work. Section III presents the system model. Section IV quantifies the system performance, and Section V details the algorithm design. Section VI simulates our results. Section VII concludes the paper.

II. Related Work & Background

A. Non-Sharding Blockchain for IoT

A convergence of blockchain and IoT is referred as IoT blockchain [1], which can be found in various fields such as smart grid, industrial IoT (IIoT), intelligent transportation, and mobile crowdsensing [12]. In an IoT blockchain, the decentralized and tamper-resistant nature of blockchain is leveraged to address various challenges associated with IoT, such as data integrity, security, interoperability, and trust among devices. Dynamic mobility and heterogeneity, as the important nature in the IoT blockchain, arouse public discussions and attention. For example, Misra et al. [13] believe that issues of the heterogeneity in networks and devices, along with the mobility should be considered when applying the blockchain in real-world IoT networks. Subsequently, Feng et al. [14] provide a dynamic selection of consensus nodes adapting to the dynamic changes of blockchain network in 5G-enabled IIoT. Liu et al. [15] devise an anti-heterogeneity consensus algorithm to scale IoT blockchain. However, all works have ignored that the structure of single-chain in non-sharding blockchain leads to poor scalability, which is a huge challenge for extending large-scale IoT blockchain application.

B. Static and Homogeneous Sharding for IoT

In the context of blockchain, sharding is a technique used to address the scalability challenges inherent in traditional blockchain architecture. By implementing sharding in a blockchain, the network is partitioned into smaller committees or shards. Each shard operates independently and processes its own set of transactions. This allows multiple transactions to be processed concurrently across different shards, significantly increasing the overall throughput of the blockchain network. For instance, OmniLedger [6] demonstrates a state sharding blockchain system, in which the whole ledger is divided into several subsets that are separately maintained by each shard, thereby reducing the cost of storage. Then, Zamani et al. [5] propose RapidChain to deal with the high overhead issue of sharding reconfiguration in OmniLedger. Wang et al. [7] design Monoxide, in which the asynchronous consensus zones are proposed to scale the system linearly without lowering the security or decentralization. Recently, Li et al. [9] present a polynomially coded sharding scheme named PolyShard. This scheme can realize efficient scaling on storage and throughput without compromising on trust.

Inspired by the advances, some further efforts introduce sharding to the IoT blockchain. The work like MasterShards [16], which provides a distributed state update sharding architecture for the IoT devices with limited computing resources. Liu et al. [17] extend the use of sharding for large-scale SDN networks. Xue et al. [18] use sharding to construct a scalable blockchain-based spectrum system based on OmniLedger [6]. However, most sharding are assumed to be static and homogeneous, e.g., partition and reconfigure shards in a random and fixed manner, which is impractical to the IoT blockchain featured with dynamics and heterogeneity.

C. Dynamic Sharding for IoT

Exploiting the DRL to implement dynamic sharding in IoT blockchain is complex and challenging. Yun et al. [19] use a deep-Q-network (DQN) to develop a sharding framework to achieve the optimal throughput and security level for a dynamic IoT environment with malicious attacks. Yuan et al. [20] formulate the dynamic adjustments of shard number and consensus parameter as a MDP, then train them with a double deep-Q-network (DDQN) for collaborative computing in the IoT. Similarly, Yang et al. [21] use the DRL to select shard number, block size, and block generation interval for the dynamic mobile edge computing system. However, reconfiguration, as one of the most critical components for dynamic sharding to guarantee the security of IoT blockchain [3], none of them have strictly consider and quantify it at each epoch in their model.

Furthermore, IoT devices are assumed to travel to new environments, thus the DRL agent needs to be relearned frequently which requires huge amounts of data and consumes long-term training to reach reasonable performance. This is unacceptable because high latency will limit the applicability of the IoT blockchain to massive real-world applications. Through this work, we attempt to handle these issues and develop an improved DRL algorithm to accelerate the training process of agents with limited amounts of data.

III. System Model

SmartChain considers a sharded IoT blockchain system, which consists of several fixed base stations and a large number of dynamic mobile and heterogeneous IoT devices. Here we implement storage sharding in which each shard maintains an independent disjoint ledger. As illustrated in Fig. 3, this system typically proceeds in *epochs*, each of which includes two phases, i.e., consensus and reconfiguration. At the end of any epoch t, IoT nodes' new location coordinates and varied amount of computing power are sent to DRL agents. A DRL agent is the leader of each shard selected at epoch t, which will make sharding policy based on the current input state. Afterward, all of the shard leaders constitute a leader committee (denoted by L_K) to work collaboratively agreeing on a new sharding policy for the next epoch $t+1$ via a Byzantine consensus protocol. Based on the agreement, IoT blockchain can be reconfigured and enter the new epoch $t+1$. Next gives the relevant system models.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

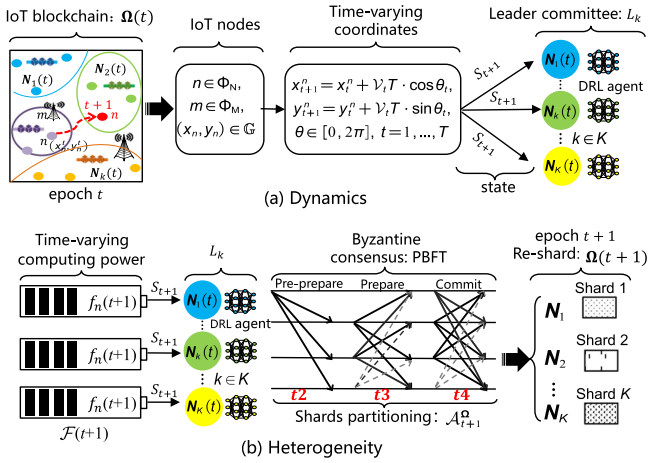


Fig. 3. Sharding decisions for SmartChain across an epoch.

A. Consensus Model

We adopt a hybrid consensus mechanism like most sharding systems [4], [5], [6], [7], [8]. For security, we use the Nakamoto consensus protocol, such as Proof-of-Work (PoW) to establish IoT node identities. Then, we use more energy-efficient Practical Byzantine Fault Tolerance (PBFT) protocol to make consensus on blocks. The PBFT is also used to reach a consensus for each shard and the leadership committee. In our model, we set the shard Byzantine fault tolerance $f = \frac{1}{3}$ and the system Byzantine fault tolerance $f_s = \frac{1}{4}$. This is because our previous work [10] has proved that the probability of formation of an unsafe shard will be less than 10% when the system's malicious nodes are less than one fourth of the total nodes, i.e., $f_s = \frac{1}{4}$. The consensus procedure of PBFT contains five phases, i.e., *Request*, *Pre-prepare*, *Prepare*, *Commit*, and *Replay*. In each shard, one IoT device is selected as *primary* while others are *replicas*. More details about the message delivery between *primary* and *replicas* can refer to *Castro*'s work [22].

B. Sharding Model

We model the network wherein the base stations are defined as a set $\Phi_{\mathrm{M}} \triangleq \{m = 1, 2, \ldots, M\}$, which are randomly located at fixed positions. The IoT devices are denoted by a set $\Phi_{\mathrm{N}} \triangleq \{n = 1, 2, \ldots, N\}$, each of which is initially distributed at an independent random position in $\mathbb{G}$. The geographical location of IoT device n is represented by a two-dimensional coordinate $(x_n, y_n) \in \mathbb{G}$, where $\mathbb{G} \triangleq \{(x_n, y_n)\}_{n \in \Phi_{\mathrm{N}}}$ is the set of location coordinates. Note that, the blockchain nodes can be any mobile IoT devices, which are assumed to move in a slotted fashion, see e.g., [23]. Let the speed of IoT device n be $\mathcal{V}$ (in meter/slot), n's location coordinate at any time slot t ($t \in \boldsymbol{T}$) can be found by $x^n_{t+1} = x^n_t + \mathcal{V}_t T \cdot \cos\theta_t$ and $y^n_{t+1} = y^n_t + \mathcal{V}_t T \cdot \sin\theta_t$, where $\mathcal{V}$ is a random variable with a Normal distribution, θ has a Uniform distribution over the interval $[0, 2\pi]$, and T is the operational time of the system during an entire time slot.

For any epoch t, we assume that there exist K shards, each of which runs a PBFT consensus protocol to agree on the block. This system can be denoted by $\boldsymbol{\Pi}(t) = \{\boldsymbol{N}_1(t), \ldots, \boldsymbol{N}_k(t), \ldots, \boldsymbol{N}_K(t)\} \in \boldsymbol{\Omega}(t)$, which indicates the set of IoT devices Φ_{N} is partitioned into K separated shards where $k \in \{1, 2, \ldots, K\}$. The number of IoT devices in each shard $|\boldsymbol{N}_k|$ are bounded by $|\boldsymbol{N}_k| \geq 4$, and $\boldsymbol{\Omega}$ is the collection of shards which can be modeled by

$$\boldsymbol{\Omega} \triangleq \{\boldsymbol{\Pi} = \{\boldsymbol{N}_1, \boldsymbol{N}_2, \ldots, \boldsymbol{N}_K\} \mid |\boldsymbol{N}_k| \geq 4,$$
$$\boldsymbol{N}_k \cap \boldsymbol{N}_j = \oslash, \forall k, j \in \{1, 2, \ldots, K\}, \bigcup\nolimits_{k=1}^{K} |\boldsymbol{N}_k| = N\}. \tag{1}$$

C. Communication Model

We assume the network is partially synchronous, i.e., any communication message could be received within a bounded time interval $\mathcal{T}$. An IoT device n relies on the base station m to communicate. Since Shannon's theory [24] provides a mathematical framework for quantifying and studying information transmission and communication systems, we use the Shannon's theory to model the data transmission rate $R_{n,m}$ as follows,

$$R_{n,m} = B \cdot \log_2\left(1 + \frac{h_n g_n}{\sum_{n' \in \Phi_{\mathrm{N}} \backslash \{n\}} h_{n'} g_{n'} + \delta^2}\right), \tag{2}$$

where B is channel bandwidth, h_n is transmit power, g_n is channel gain, and δ^2 is background noise variance. The term $\Phi_{\mathrm{N}} \backslash \{n\}$ is the set of other IoT devices associated to m, which share the same channel and create interferences [25].

Given that the data transmission rate between any two IoT devices is a time-varying variable due to the devices' mobility, we model the transition of random variable $R_{n_i,n_j}(t)$ as a Markov chain, see e.g., [23], where $R_{n_i,n_j}(t)$ denotes the transmission rate of the link connecting two IoT devices n_i and n_j at time slot $t, i, j = 1, 2, \ldots, N$. The data transmission rate can be quantized into L discrete intervals with $\mathcal{R} = \{\mathcal{R}_0, \mathcal{R}_1, \ldots, \mathcal{R}_{L-1}\}$. Based on $L \times L$ state transition probability matrix $\boldsymbol{\Upsilon}(t)$, the data transmission rates change from one state to another, which can be denoted as

$$\boldsymbol{\Upsilon}(t) = \left[Pr\left(R_{n_i,n_j}(t+1) = b_s | R_{n_i,n_j}(t) = a_s\right)\right]_{L \times L}, \tag{3}$$

where $a_s, b_s \in \mathcal{R}$.

D. Computation Model

The heterogeneity of computing power is an essential feature of the IoT blockchain. Facing a total of N IoT devices, we cannot exactly know the amount of computing power employed for device n at the next time instant. Thus, n's computing power can be modeled as a random variable f_n, which can be divided into X discrete intervals, written as $\mathcal{F} = \{\mathcal{F}_0, \mathcal{F}_1, \ldots, \mathcal{F}_{X-1}\}$. That is, n's available computing power at time slot t is $f_n(t)$. Considering the time correlation of the computation power, we model the transition of computing power deployed on each IoT device as a Markov chain. The $X \times X$ state transition probability matrix $\boldsymbol{\Theta}^n(t)$ is expressed as

$$\boldsymbol{\Theta}^n(t) = \left[Pr\left(f_n(t+1) = \omega_s | f_n(t) = \upsilon_s\right)\right]_{X \times X}, \tag{4}$$

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

where $\upsilon_s, \omega_s \in \mathcal{F}$.

E. Cyber Attack Model

There are two kinds of devices and base stations: *honest* and *malicious*. The honest are supposed to abide by all system's adopted protocols. Whereas the malicious may deny services, or collude with each other to vote, thus rejecting commit results from an honest block but accepting results from the faulty block created by malicious devices.

We assume the attack of Byzantine adversary is slowly-adaptive [5], [6], [7], which is often used in the analysis and modeling of security protocols in the context of distributed systems and consensus algorithms. This model has advantages in terms of detection and stability. Specifically, the adversaries are assumed to focus their power to corrupt honest IoT devices or even the honest base station, but it will take a certain amount of time. Assume that a successful attack can cause the device or base station to become malicious nodes. We denote the computing power of adversaries by ξ and the computing power of IoT device n ($n \in \Phi_{\mathrm{N}}$) employed for cyber self-defense by f_n. Inspired by [26], [27], the probability of a successful cyber-attack defense for IoT device n can be represented by

$$p_n = \frac{f_n}{f_n + \xi} = 1 - \frac{\xi}{f_n + \xi}. \tag{5}$$

Either a successful attack on the IoT device or its connected base station can cause a faulty device n. Thus, the probability of faulty device n is

$$\dot{p_n} = 1 - p_n \cdot p_{m_n}, \tag{6}$$

where p_{m_n} represents the probability of a successful cyber-attack defense for n's associated base station m_n.

IV. Quantitative Analyses of Sharding Performance

A. Throughput

The throughput of SmartChain directly relates to four crucial parameters: (1) consensus rounds, denoted by r, in each of which a shard proposes one block; (2) block size, denoted by $\mathcal{S}^B$; (3) average transaction size, denoted by $\mathcal{S}^T$; and (4) epoch latency, denoted by T_{epoch}, which is the sum of consensus latency and reconfiguration latency. We consider the throughput of shard as in [10], [19], [21], [23], which is common for quantifying sharded blockchains. Thus, the transaction throughput for each shard $\boldsymbol{N}_k$ processed in an epoch can be defined as

$$\mathbb{T}_k\left(\boldsymbol{N}_k\right) = \frac{r \cdot \mathcal{S}^B / \mathcal{S}^T}{T_{epoch}}, \tag{7}$$

where T_{epoch} is computed by

$$T_{epoch} = \max_{k \in \{1,\ldots,K\}} \left(r \cdot T_k^{con}\left(\boldsymbol{N}_k\right) + T_{\mathbf{\Pi}}^{reco}\right). \tag{8}$$

T_{epoch} is the sum of consensus latency $r \cdot T_k^{con}(\boldsymbol{N}_k)$ and reconfiguration latency $T_{\mathbf{\Pi}}^{reco}$. The term $T_k^{con}(\boldsymbol{N}_k)$ stands for a round of consensus time, so the total consensus latency in an epoch is computed as $r \cdot T_k^{con}(\boldsymbol{N}_k)$. To ensure that all shards of the system can be synchronized to come into the epoch reconfiguration, we assume T_{epoch} is determined by the slowest shard with the largest latency. Therefore, the throughput of system is

$$\mathbb{T}^{total} = \sum_{k=1}^{K} \mathbb{T}_k\left(\boldsymbol{N}_k\right). \tag{9}$$

Observing (7)–(9), we find that increasing $\mathcal{S}^B$ or decreasing $\mathcal{S}^T$ can improve $\mathbb{T}^{total}$. Additionally, two metrics (i.e., consensus latency $T_k^{con}(\boldsymbol{N}_k)$ and reconfiguration latency $T_{\mathbf{\Pi}}^{reco}$) are also of great importance directly related to $\mathbb{T}^{total}$. In the following, we quantify these two factors in detail.

B. Consensus Latency

For quantitative analysis of PBFT, as with many other studies (e.g., [19], [23], [28]), we regard the consensus latency as a total time of message propagation and verification. For message propagation, we model the data transmission rate $R_{n_i,n_j}(t)$ as a finite-state Markov channel model (see Section III-C), where $R_{n_i,n_j}(t)$ is quantized into L levels with $\mathcal{R} = \{\mathcal{R}_0, \mathcal{R}_1, \ldots, \mathcal{R}_{L-1}\}$. For message verification, we assume that verifying one signature needs α CPU cycles, generating and verifying one message authentication code (MAC) require β CPU cycles when computing the cost of cryptographic operations. Accordingly, we denote the latency of each consensus $T_k^{con}(\boldsymbol{N}_k)$ for shard k as the sum of message propagation time T_{prop}^k and message verification time T_{val}^k, i.e.,

$$T_k^{con}\left(\boldsymbol{N}_k\right) = T_{prop}^k + T_{val}^k. \tag{10}$$

Next will make a quantitative analysis of these two factors. First, we quantify the latency on message verification. Since the validation of message within a shard is performed by both primary and replicas in parallel, the message verification time T_{val}^k for the shard is determined by the slowest node that has the largest delay. Therefore, T_{val}^k can be expressed as

$$T_{val}^k = \frac{1}{\mathcal{M}} \max_{i \in \{1,\ldots,|\boldsymbol{N}_k|-1\}} \left(T_{primary}^k, T_{replica}^i\right), \tag{11}$$

where $\mathcal{M}$ denotes the batch size of blocks, $T_{primary}^k$ and $T_{replica}^i$ indicate the time of primary and replicas for verifying requested messages in shard k, respectively. Specifically, the message verification time $T_{primary}^k$ for primary in the kth shard is given by

$$T_{primary}^k = \frac{\mathcal{M}\alpha + \left[2\mathcal{M} + 4\left(|\boldsymbol{N}_k| - 1\right)\right]\beta}{f_{k,p}}, \tag{12}$$

and the message verification time $T_{replica}^i$ for each IoT replica in the kth shard is

$$T_{replica}^i = \frac{\mathcal{M}\alpha + \left[\mathcal{M} + 4\left(|\boldsymbol{N}_k| - 1\right)\right]\beta}{f_{k,i}}, \tag{13}$$

where $f_{k,p}$ and $f_{k,i}$ are the computing power of primary and IoT replica within shard k, respectively.

Second, we evaluate the latency of message propagation. Similar to [19], [20], [23], [28], we mainly consider three-phase (i.e., pre-prepare, prepare, and commit) when computing the message propagation time T_{prop}^k for each request within a shard.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

Since the message deliveries of IoT devices are performed in parallel, we have

$$\begin{aligned} T_{prop}^{k} &= \frac{1}{\mathcal{M}}\left(t_2^k + t_3^k + t_4^k\right) \\ &= \frac{1}{\mathcal{M}}\Big(\min\left\{\max_{i\neq p} T_{n_{k,p},n_{k,i}}, \mathcal{T}\right\} \\ &\quad + \min\left\{\max_{i\neq p,j} T_{n_{k,i},n_{k,j}}, \mathcal{T}\right\} \\ &\quad + \min\left\{\max_{i\neq j} T_{n_{k,i},n_{k,j}}, \mathcal{T}\right\}\Big), \end{aligned} \tag{14}$$

where t_2^k, t_3^k, and t_4^k denote the propagation time of each request for pre-prepare, prepare, and commit within each shard $k, k \in \{1, 2, \ldots, K\}$, respectively. $T_{n_{k,p},n_{k,i}}$ represents the message propagation time between the primary and an IoT replica, and $T_{n_{k,i},n_{k,j}}$ denotes the message propagation time for any two IoT devices. Note that, any message propagation is limited within a timeout $\mathcal{T}$.

As defined in Section III, we denote the data transmission rate for IoT device n by $R_{n,m}$, where $m, m \in \Phi_{\mathrm{M}}$ is the base station associated with n. Given that IoT devices connect to the base station for communication, the transmission time T_{n_1,n_2} for each communication-node pair (n_1, n_2) is

$$T_{n_1,n_2} = \frac{\mathcal{MS}^B}{R_{n_1,m_1}} + \frac{\mathcal{MS}^B}{R_{m_1,m_2}} + \frac{\mathcal{MS}^B}{R_{n_2,m_2}}, \tag{15}$$

where $\mathcal{S}^B$ represents the size of block.

C. Reconfiguration Latency

We presents a shard reconfiguration mechanism based on our previous work [10], which contains a five-stage procedure as follows.

1) *Epoch randomness generation (T_{rand}):* Since members for each epoch require to re-elect via solving a PoW puzzle, generating epoch randomness $rand$ is the first stage. To generate $rand$, there are public-verifiability, unbiasability, availability, and unpredictability approaches, including verifiable random function (VRF) [29], verifiable delay function (VDF) [30], and trusted execution environment (TEE) [31]. Similar to RapidChain [5], we assume a reference committee C_R is responsible for generating $rand$. Particularly, we use the set of base stations to act as C_R. Thus, T_{rand} is the cost of agreements on $rand$ made by C_R, i.e., T_{Cr}^{con}.

2) *Creations of state blocks (T_s):* The state block stores the latest information of transactions for each shard, like the account addresses and states (e.g., balance, nonce, and storage root) [10]. To prevent the loss of shard information during reconfiguration, primary could create the state block of local shard for the epoch by running an intra-shard consensus protocol; then, sending the state blocks to C_R. In this stage, T_s is determined by the slowest intra-shard consensus with the highest delay, i.e., $\max_{\boldsymbol{N}_k \in \boldsymbol{\Pi}} T_k^{con}(\boldsymbol{N}_k)$.

3) *New identity establishment (T_{id}):* To join in a new epoch, IoT devices need to commit their identity (e.g., public key, IP address, and offline PoW solutions) to C_R. Afterward, C_R checks the identities and runs the PBFT to agree on a reference block, indicating identified legal IoT devices for new epoch. Consequently, T_{id} can be denoted by the consensus time of C_R, i.e., T_{Cr}^{con}.

4) *DRL-based reconfiguration decision (T_{drl}):* SmartChain reallocates IoT devices during reconfiguration according to the decisions agreed by the leader committee, L_K. L_K is comprised of the primary of each shard. One primary $L^p_{\boldsymbol{N}^i}(L^p_{\boldsymbol{N}^i} \in L_K)$ is selected from L_K, working as a proposer to generate a sharding policy based on the state of system environments. Once the policy is proposed, it should make effect via an intra-leader-committee consensus. Thus, T_{drl} is determined by L_K's consensus, i.e., T_{Lk}^{con}.

5) *Ledgers merging and splitting (T_r):* Due to the dynamic mobility of IoT devices, the blockchain node of some shards may be fully changed. We present to maintain the information of separated ledgers for each shard by merging and splitting ledgers, i.e., first, C_R merges the header of pre-stored state blocks; then, C_R splits the merged ledger into K disjoint sets (i.e., new state blocks), where K is the shard number generated by DRL agents; finally, after an intra-reference-committee consensus, C_R broadcasts new state blocks so that IoT devices can synchronize the information of their shards. Accordingly, T_r can be calculated as the consensus time of C_R, i.e., T_{Cr}^{con}.

Totally, the latency of reconfiguration $T_{\boldsymbol{\Pi}}^{reco}$ is

$$\begin{aligned} T_{\boldsymbol{\Pi}}^{reco} &= T_{rand} + T_s + T_{id} + T_{drl} + T_r \\ &= T_{Cr}^{con} + \max_{\boldsymbol{N}_k \in \boldsymbol{\Pi}} T_k^{con}(\boldsymbol{N}_k) + T_{Cr}^{con} + T_{Lk}^{con} + T_{Cr}^{con} \\ &= 3T_{Cr}^{con} + \max_{\boldsymbol{N}_k \in \boldsymbol{\Pi}} T_k^{con}(\boldsymbol{N}_k) + T_{Lk}^{con}. \end{aligned} \tag{16}$$

Based on (10)–(14), T_{Cr}^{con} can be derived as follows,

$$\begin{aligned} T_{Cr}^{con} &= T_{prop}^{Cr} + T_{val}^{Cr} = \frac{1}{\mathcal{M}}\left(t_2^m + t_3^m + t_4^m\right) \\ &\quad + \frac{1}{\mathcal{M}} \max_{i\in\{1,\ldots,M-1\}} \left(T_{primary}^{m_p}, T_{replica}^{m_i}\right) \\ &= \frac{1}{\mathcal{M}}\Big(\min\left\{\max_{i\neq p} T_{m_p,m_i}, \mathcal{T}\right\} \\ &\quad + \min\left\{\max_{i\neq j\neq p} T_{m_i,m_j}, \mathcal{T}\right\} \\ &\quad + \min\left\{\max_{i\neq j} T_{m_i,m_j}, \mathcal{T}\right\}\Big) \\ &\quad + \frac{1}{\mathcal{M}} \max_{i\in\{1,\ldots,M-1\}} \left(\frac{\mathcal{M}\alpha + [2\mathcal{M} + 4(M-1)]\beta}{f_{m_p}}, \right. \\ &\quad \left. \frac{\mathcal{M}\alpha + [\mathcal{M} + 4(M-1)]\beta}{f_{m_i}} \right), \end{aligned} \tag{17}$$

where f_{m_p} and f_{m_i} denote the available computing power of primary and replicas in the reference committee $C_R, C_R \in \{1, 2, \ldots, M\}$, respectively. m_p, m_i, and m_j represent different base stations, where p is primary while i, j are replicas. The term $\max_{\boldsymbol{N}_k \in \boldsymbol{\Pi}} T_k^{con}(\boldsymbol{N}_k)$ indicates the slowest consensus latency on state-block agreements for shard k that with the largest time. According to (10), $T_k^{con}(\boldsymbol{N}_k)$ is denoted as the sum of

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

message propagation time T^k_{prop} and message verification time T^k_{val}. Therefore, $\max_{\boldsymbol{N}_k\in\boldsymbol{\Pi}} T^{con}_k(\boldsymbol{N}_k)$ can be written as

$$\begin{aligned}\max_{\boldsymbol{N}_k\in\boldsymbol{\Pi}} T^{con}_k(\boldsymbol{N}_k) &= \max_{\boldsymbol{N}_k\in\boldsymbol{\Pi}} \left(T^k_{prop}+T^k_{val}\right)\\ &= \max_{\boldsymbol{N}_k\in\boldsymbol{\Pi}} \Bigg(\frac{1}{\mathcal{M}}\left(t^k_2+t^k_3+t^k_4\right)+\frac{1}{\mathcal{M}}\\ &\quad\times \max_{i\in\{1,\ldots,|\boldsymbol{N}_k|-1\}}\left(T^k_{primary},T^i_{replica}\right)\Bigg)\\ &= \max_{\boldsymbol{N}_k\in\boldsymbol{\Pi}} \Bigg(\frac{1}{\mathcal{M}}\Big(\min\left\{\max_{i\neq p} T_{n_{k,p},n_{k,i}},\mathcal{T}\right\}\\ &\quad+\min\left\{\max_{i\neq p,j} T_{n_{k,i},n_{k,j}},\mathcal{T}\right\}\\ &\quad+\min\left\{\max_{i\neq j} T_{n_{k,i},n_{k,j}},\mathcal{T}\right\}\Big)\\ &\quad+\frac{1}{\mathcal{M}}\max_{i\in\{1,\ldots,|\boldsymbol{N}_k|-1\}}\\ &\quad\times\Bigg(\frac{\mathcal{M}\alpha+\left[2\mathcal{M}+4\left(|\boldsymbol{N}_k|-1\right)\right]\beta}{f_{k,p}},\\ &\qquad\frac{\mathcal{M}\alpha+\left[\mathcal{M}+4\left(|\boldsymbol{N}_k|-1\right)\right]\beta}{f_{k,i}}\Bigg)\Bigg),\end{aligned} \tag{18}$$

where $f_{k,p}$ and $f_{k,i}$ is the amount of computing power of primary and IoT replicas in shard $k, k\in\{1,2,\ldots,K\}$, respectively. $|\boldsymbol{N}_k|$, $\boldsymbol{N}_k\in\boldsymbol{\Pi}$ is the number of IoT devices in shard. The term T^{con}_{Lk} denotes the consensus latency for the leader committee L_K during reconfiguration. Then, we denote T^{con}_{Lk} by

$$\begin{aligned}T^{con}_{Lk} &= T^{Lk}_{prop}+T^{Lk}_{val} = \frac{1}{\mathcal{M}}\left(t^{\boldsymbol{N}_{k,p}}_2+t^{\boldsymbol{N}_{k,p}}_3+t^{\boldsymbol{N}_{k,p}}_4\right)\\ &\quad+\frac{1}{\mathcal{M}}\max_{j\in\{1,\ldots,K-1\}}\left(T^{\boldsymbol{N}_{i,p}}_{primary},T^{\boldsymbol{N}_{j,p}}_{replica}\right)\\ &= \frac{1}{\mathcal{M}}\Bigg(\min\left\{\max_{j\neq i} T_{\boldsymbol{N}_{i,p},\boldsymbol{N}_{j,p}},\mathcal{T}\right\}\\ &\quad+\min\left\{\max_{j\neq i,u} T_{\boldsymbol{N}_{j,p},\boldsymbol{N}_{u,p}},\mathcal{T}\right\}\\ &\quad+\min\left\{\max_{j\neq u} T_{\boldsymbol{N}_{j,p},\boldsymbol{N}_{u,p}},\mathcal{T}\right\}\Bigg)\\ &\quad+\frac{1}{\mathcal{M}}\max_{j\in\{1,\ldots,K-1\}}\Bigg(\frac{\mathcal{M}\alpha+\left[2\mathcal{M}+4\left(K-1\right)\right]\beta}{f_{\boldsymbol{N}_{i,p}}},\\ &\qquad\frac{\mathcal{M}\alpha+\left[\mathcal{M}+4\left(K-1\right)\right]\beta}{f_{\boldsymbol{N}_{j,p}}}\Bigg),\end{aligned} \tag{19}$$

where $\boldsymbol{N}_{k,p}$ ($\boldsymbol{N}_{k,p}\in\boldsymbol{\Pi}$) stands for the primary of shard $\boldsymbol{N}_k$. $\boldsymbol{N}_{i,p}$, $\boldsymbol{N}_{j,p}$, and $\boldsymbol{N}_{u,p}$ represent the primary, replicas, and any IoT device in the leader group L_K, respectively. The terms $f_{\boldsymbol{N}_{i,p}}$ and $f_{\boldsymbol{N}_{j,p}}$ are the available computing power, respectively for the primary and replicas in L_K.

D. Security

Recall that the limited Byzantine fault tolerance within each shard, the ratio of malicious devices number to the size of shard is a top concern. In SmartChain, there will be a security penalty $\mathcal{P}_k(\boldsymbol{N}_k)$, which measures how safe it is to form a shard with the proportion of honest/malicious IoT devices in relation to the total number of sharding nodes. According to [32], the uneven distribution of malicious devices can cause the failure of shards. We consider the uneven distribution of the defensive failure (i.e. malicious) IoT devices in cyber attacks, as well as the successful defensive (i.e., honest) IoT devices, thus the security penalty of each shard $\boldsymbol{N}_k$ is expressed as

$$\mathcal{P}_k\left(\boldsymbol{N}_k\right)=e^{\left|N^f_k(\boldsymbol{N}_k)-\bar{N}^f\right|+\left|N^s_k(\boldsymbol{N}_k)-\bar{N}^s\right|}, \tag{20}$$

where $N^f_k(\boldsymbol{N}_k)$ and $N^s_k(\boldsymbol{N}_k)$ are expected values of the total number respectively for malicious and honest IoT devices within shard k. $\bar{N}^f$ and $\bar{N}^s$ are expected values of the average number respectively for malicious and honest IoT devices in the system. Through (20), we can scatter malicious devices across the shards as evenly as possible by minimizing $\mathcal{P}_k(\boldsymbol{N}_k)$. Based on (5) and (6), $N^f_k(\boldsymbol{N}_k)$ can be calculated by

$$N^f_k\left(\boldsymbol{N}_k\right)=\sum_{n\in\boldsymbol{N}_k}\dot{p}_n=\sum_{n\in\boldsymbol{N}_k}\left(1-\frac{f_n}{f_n+\xi}\cdot\frac{f_{m,n}}{f_{m,n}+\xi}\right), \tag{21}$$

where f_n and $f_{m,n}$ indicate the computing power employed to defense adversaries, respectively for IoT device n and the corresponding base station m_n. And ξ represents the computing power owned by malicious attackers to attack n. The term $N^s_k(\boldsymbol{N}_k)$ indicates the expected value of the total number of IoT devices for a successful defense in each shard. Based on (5), we define $N^s_k(\boldsymbol{N}_k)$ as

$$N^s_k\left(\boldsymbol{N}_k\right)=\sum_{n\in\boldsymbol{N}_k}p_n=\sum_{n\in\boldsymbol{N}_k}\frac{f_n}{f_n+\xi}. \tag{22}$$

Accordingly, the expected value of the average number of malicious IoT devices in the system is given by

$$\bar{N}^f=\frac{\sum_{n\in\boldsymbol{\Pi}}\dot{p}_n}{K}=\frac{1}{K}\cdot\sum_{n\in\boldsymbol{\Pi}}\left(1-\frac{f_n}{f_n+\xi}\cdot\frac{f_{m,n}}{f_{m,n}+\xi}\right), \tag{23}$$

where $\boldsymbol{\Pi}=\{\boldsymbol{N}_1,\boldsymbol{N}_2,\ldots,\boldsymbol{N}_K\}$ denotes the collection of disjoint shards for a whole system, and K is the shard number. Then, the expected value of the average number of successful defensive IoT devices for the system is written as

$$\bar{N}^s=\frac{\sum_{n\in\boldsymbol{\Pi}}p_n}{K}=\frac{1}{K}\cdot\sum_{n\in\boldsymbol{\Pi}}\frac{f_n}{f_n+\xi}. \tag{24}$$

V. Algorithm Design

In this section, we detail the design of TPPOD. First, we formulate the sharding decision with a long-term tradeoff between throughput and security as a MDP. To solve this problem, we present the network architecture of TPPOD. Then, we describe the training process. Finally, we present a distributed agents deployment and consensus scheme.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

A. MDP Formulation

1) State Space: We consider a dynamic and heterogeneous system with N devices, where the moving of IoT devices frequently happens and the computing power employed for security defense varies with different environments. Thus, the state is defined as a union of the geographical location of IoT devices $\mathbb{G}(t) = \{(x_n(t), y_n(t))|_{n\in\Phi_N}\}$, the data transmission rate between IoT devices and their linked base stations $\boldsymbol{\Upsilon}(t) = \{\Upsilon_m^n(t)|_{n\in\Phi_N, m\in\Phi_M}\}$, and the available computing power $\mathbb{F}(t) = \{f^n(t)|_{n\in\Phi_N}\}$, which can be defined as

$$\mathcal{S}(t) = \begin{bmatrix} \mathcal{G}^1(t) & \mathcal{G}^2(t) & \cdots & \mathcal{G}^N(t) \\ \Upsilon_m^1(t) & \Upsilon_m^2(t) & \cdots & \Upsilon_m^N(t) \\ f^1(t) & f^2(t) & \cdots & f^N(t) \end{bmatrix}. \tag{25}$$

2) Action Space: By observing the state, DRL agents make decisions about the number of shards, partition structure (i.e., which IoT devices are allocated to the same shard), and the primary selection mode (i.e., the selected mode of primary at current epoch). Thus, the action is

$$\mathcal{A}(t) = \left\{k, \mathcal{A}^{\Omega}(t), \mathcal{A}^{P}(t)\right\}, \tag{26}$$

where $k \in \{1, 2, \ldots, K\}$ indicates the number of shards at decision epoch $t, t \in \{1, 2, \ldots\}$. Here, $k \in [1, \frac{N}{4}]$, and $k = 1$ denotes there is no sharding implemented. We set $k = \frac{N}{4}$ to give the maximum allowable shard number, where N is the number of IoT devices for epoch t.

$\mathcal{A}^{\Omega}(t) = \{\boldsymbol{N}^1(t), \boldsymbol{N}^2(t), \ldots, \boldsymbol{N}^i(t), \ldots, \boldsymbol{N}^K(t)\}$ means the partition structure, i.e., the shard set $\boldsymbol{\Pi}(t)$. The term $\boldsymbol{N}^i(t)$, $\boldsymbol{N}^i(t) \in \boldsymbol{\Pi}(t)$ indicates which IoT devices are assigned to the same shard i, $i \in \{1, 2, \ldots, K\}$ at epoch t, and there is $\boldsymbol{N}^i(t) = \{n_i^1(t), n_i^2(t), \ldots, n_i^j(t), \ldots, n_i^{|\boldsymbol{N}^i(t)|}\}$. Here, $n_i^j(t)$ stands for the j-th IoT device in the i-th shard, and $|\boldsymbol{N}^i(t)|$ is the number of IoT devices in the i-th shard. This term satisfies $|\boldsymbol{N}^i(t)| \geq 4$ under security guarantees of PBFT and there is $\sum_{i=1}^{K} |\boldsymbol{N}^i(t)| = N$.

$\mathcal{A}^{P}(t) = \{a^1(t), a^2(t), \ldots, a^i(t), \ldots, a^K(t)\}$ determines which primary selection mode is conducted. We provide a two-mode selection $a^i(t) \in \{0, 1\}$, where $a^i(t) = 1$ indicates shard i will select primary by considering the location state $\mathbb{G}(t)$ and the transmission state $\boldsymbol{\Upsilon}(t)$; otherwise, $a^i(t) = 0$ means to select primary via the state of available computing power, i.e., $\mathbb{F}(t)$. Note that only one mode is adopted at one-time slot.

3) System Reward Function: We consider an optimization problem making a trade-off between throughput and security for the sharded IoT blockchain system, which can be decomposed into two objectives as

$$Objective\ 1: \quad \max_{\boldsymbol{\Pi}\in\boldsymbol{\Omega}} \sum_{k=1}^{K} \mathbb{T}_k(\boldsymbol{N}_k) \tag{27}$$

$$Objective\ 2: \quad \min_{\boldsymbol{\Pi}\in\boldsymbol{\Omega}} \sum_{k=1}^{K} \mathcal{P}_k(\boldsymbol{N}_k) \tag{28}$$

$$\text{subject to} \quad 4 \leq |\boldsymbol{N}_k| \leq N, \ \forall N \in \Phi_N, \tag{29}$$

$$\sum_{1}^{K} |\boldsymbol{N}_k| = N, \ k \in \{1, 2, \ldots, K\}, \tag{30}$$

$$2 \leq k \leq \frac{N}{4}, \ \forall k \in \{1, 2, \ldots, K\}. \tag{31}$$

Constraints (29), (30) imply that the number of IoT devices for each shard, i.e., $|\boldsymbol{N}_k|$, ranges from $[4, N]$; the sum of $|\boldsymbol{N}_k|$ is equal to the total number of all devices in IoT blockchain. Constraint (31) suggests that the system should be partitioned with a maximum allowable shard number, which cannot exceed one-quarter of the total number of all IoT devices.

The system reward is the combination of $Objective\ 1$ and $Objective\ 2$, aiming to maximize the system reward by balancing throughput and security for each epoch. Then, we specify the system reward $\mathcal{R}(t)$ in epoch t as

$$\mathcal{R}\left(\mathcal{S}(t), \mathcal{A}(t)\right) = \sum_{k=1}^{K} \mathbb{T}_k(\boldsymbol{N}_k) - \omega \sum_{k=1}^{K} \mathcal{P}_k(\boldsymbol{N}_k), \tag{32}$$

where ω is the reward priority weight. By setting different values of ω, DRL agent can optimize the system reward with different priorities between throughput and security customized for the IoT applications.

B. Proposed TPPOD Algorithm

SmartChain requires the DRL agent to learn quickly when IoT devices frequently change the environment states. This motivates us to propose a Transferable Proximal Policy Optimization (PPO) with Demonstrations algorithm, namely TPPOD, the essence of which is leveraging small sets of demonstration data to speed up learning. Such ideas are inspired by Transfer Learning (TL) [11], which provides prior knowledge that can be accessible by a target new environment, so the agent of target new environments can learn better by leveraging experience information.

Specifically, we consider two related IoT blockchain environments modeled with MDP, i.e., a source (current) environment $\mathcal{O}_s$ and a target (new) environment $\mathcal{O}_t$. TL aims to learn the optimal shard-partitioning policy π^* for $\mathcal{O}_t$ by using the demonstration data $\mathcal{D}^s$ from $\mathcal{O}_s$, as well as the experience data $\mathcal{D}^t$ from $\mathcal{O}_t$. That is,

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{s\sim\mu_t, a\sim\pi}\left[Q^{\pi}_{\mathcal{O}_t}(s, a)\right], \tag{33}$$

where μ_t is the expected value of a normal distribution for action selection. $\pi = \phi(\mathcal{D}^s \sim \mathcal{O}_s, \mathcal{D}^t \sim \mathcal{O}_t)$ is a function mapping from state $\mathcal{S}(t)$ to action $\mathcal{A}(t)$ under $\mathcal{O}_t$, which can be approximated using the deep neural network training based on $\mathcal{D}^s$ and $\mathcal{D}^t$. $\mathcal{D}^s = \oslash$ is a special case meaning a regular training without the TL. That way, policy π is purely learned from the information of new environments, i.e., $\pi = \phi(\mathcal{D}_t)$.

Fig. 4 shows the network architecture of TPPOD, i.e., $\text{actor}_{\text{new}}$ network θ', $\text{actor}_{\text{old}}$ network θ, critic network θ^c, and the prioritized-experience replay memory (PERM) $\mathcal{M}^{replay}$. The $\text{actor}_{\text{new}}$, acting as the actor policy network, is responsible for constructing a normal distribution $\mathcal{N}(\mu_{\theta',t}, \sigma^2_{\theta',t})$ of action, then interacting cyclically with the environment to generate transitions $(s_t,\ a_t,\ r_t)$. The $\text{actor}_{\text{old}}$ has the same network structure as the actor_new, thus generating the normal distribution $\mathcal{N}(\mu_{\theta,t}, \sigma^2_{\theta,t})$ constraining the learning step size

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

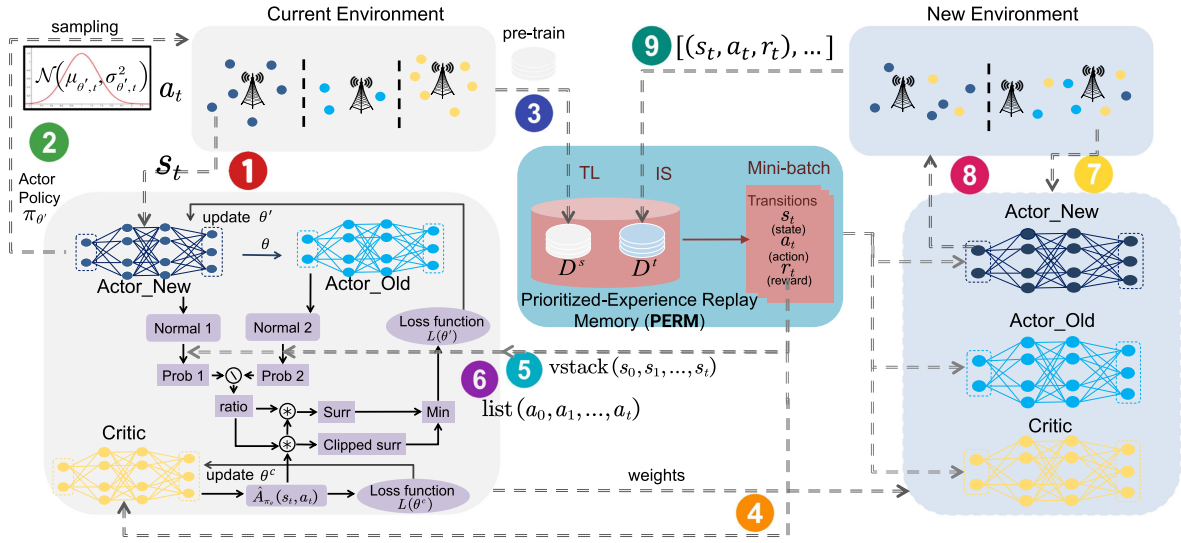


Fig. 4. Transferable Proximal Policy Optimization with Demonstrations (TPPOD) approach for dynamic and self-adaptive sharding.

of the actor_new. The critic generates the advantage estimate $\hat{A}_{\pi_\theta}(s_t, a_t)$ to train the network. The PERM is designed to store the self-generated experience data $\mathcal{D}^t$ and demonstration data $\mathcal{D}^s$.

The execution process of TPPOD in Fig. 4 involves nine phases: ① In a pre-training stage, the actor_new obtains the input state s_t from the current environment, including the IoT device's geographical location, data transmission rate, and the available computing power. Then ② it outputs an action a_t based on policy $\pi_{\theta'}$. ③ The IoT agent executes the action a_t and obtains the immediate reward r_t, the tuple (s_t, a_t, r_t) is then stored into the PERM. After a certain number of interactions, ④ a mini-batch is sampled from the PERM to calculate the loss and update the parameter of critic network. ⑤ The stored states $(s_0, s_1, \ldots, s_t)$ are combined into actor_old and actor_new for two normal distribution $Normal\ 1$ and $Normal\ 2$, and ⑥ the actions $(a_0, a_1, \ldots, a_t)$ are combined to calculate two probability $prob\ 1$ and $prob\ 2$, finally obtain the probability ratio of the new policy and the old policy. When the pre-training completes, the converged experience tuple (s_t, a_t, r_t) will be stored in the PERM as $\mathcal{D}^s$ for transfer learning. Once the dynamic environment changes, ⑦ the IoT agent observes the state from the new environment and ⑧ outputs the action, and then ⑨ stores the self-generated transitions into PERM as $\mathcal{D}^t$. During the training stage, both $\mathcal{D}^s$ and $\mathcal{D}^t$ are replayed to update the networks, which promotes the self-adaptive sharding efficiency.

Algorithm 1 details TPPOD, which can be described as follows. At first, the agent puts the replay memory $\mathcal{M}^{replay}$ with demonstration data $\mathcal{D}^s$, initializes the weights of actor-old, actor-new, and critic networks, and defines the hyper-parameters, including the number of episodes E, step count T, update frequency τ, small positive constant ϵ, and so on (Lines 1–3). Notably, $\mathcal{D}^s$ is the successful trajectories of IoT devices in pre-training under the real environment. Success trajectories refer to the convergence values of pre-training episodes and will be put in $\mathcal{M}^{replay}$ for sampling.

Algorithm 1: Transferable PPO With Demonstrations.

1 **Initialize:** replay memory $\mathcal{M}^{replay}$ with demonstration dataset $\mathcal{D}^s$;
2 **Initialize:** parameters of the Actor_Old network θ, Actor_New network θ', and Critic network θ^c ;
3 **Initialize:** episode number E, step count T, update frequency τ, a small positive constant ϵ;
4 **for** *episode* = 1, 2, ..., E **do**
5 **for** *step* = 1, 2, ..., T **do**
6 Observes the state s_t as an input to the Actor_New θ' and obtains the action a_t;
7 Execute action a_t, calculate the reward r_t according to Eq. (32) and obtain the next state s_{t+1};
8 Store (s_t, a_t, r_t) into the replay memory $\mathcal{M}^{replay}$, overwriting the oldest self-generated transition if $\mathcal{M}^{replay}$ is full;
9 **if** $\mathcal{M}^{replay}$ *is full* **then**
10 Sample a mini-batch of B transitions from $\mathcal{M}^{replay}$ with prioritization as in Eq. (34) and Eq. (35);
11 Calculate advantage estimates $\hat{A}_{\pi_\theta}(s_t, a_t)$ according to Eq. (36);
12 Calculate loss $L(\theta^c)$ as in Eq. (37), update the parameter θ^c;
13 Calculate loss $L(\theta')$ as in Eq. (38), update the parameter θ';
14 **if** t *mod* $\tau = 0$ **then**
15 Update parameter θ with $\theta \leftarrow \theta'$;
16 Update the priority of the mini-batch B by calculating $p_i^\sigma = |\hat{A}_{\pi_\theta, i}| + \epsilon$;

Before each training step, TPPOD requires the storage of $\mathcal{M}^{replay}$ to be full with trajectories $\mathcal{D}^s$ and $\mathcal{D}^t$. This can be implemented as Lines 6–8. First, agent starts acting on the system, inputs state s_t into the actor-new network and gets action a_t (Line 6). Second, based on (32), agent executes a_t, calculates

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

the reward r_t, and obtains the next state s_{t+1} (Line 7). Third, putting the experience tuples (s_t, a_t, r_t) generated from the new environment into $\mathcal{M}^{replay}$ (Line 8). The collection of $\mathcal{D}^t$ (i.e., Lines 6–8) will last until $\mathcal{M}^{replay}$ is full. Here the agent starts overwriting the old data when $\mathcal{M}^{replay}$ is full, however, $\mathcal{D}^s$ will never be over-written.

During each training step, agent samples a mini-batch of B transitions from $\mathcal{M}^{replay}$, by leveraging the prioritized-experience replay (PER) mechanism [33] (Line 10). TPPOD allows agent to control the relative sampling of $\mathcal{D}^s$ versus $\mathcal{D}^t$, where the probability of sampling a specified transition i can be proportional to its priority. We denote the probability for simpling the transition by $P(i)$, which is calculated as

$$P(i) = \frac{p_i^\sigma}{\sum_e^{e \in |\mathcal{M}^{replay}|} p_e^\sigma}, \tag{34}$$

where p_i^σ indicates the priority of transition, and $|\mathcal{M}^{replay}|$ is the storage size of replay memory. The exponent $\sigma \in [0.0, 1.0]$ defines how much prioritization is used by sampling. When $\sigma = 0$, it means to sample transition randomly. When $\sigma \neq 0$, the priority $p_i^\sigma = |\hat{A}_i| + \epsilon$, where $\hat{A}_i$ is the advantage estimate calculated for transition i, and ϵ is a small positive constant that ensures all transitions can be simpled with some probability. To deal with the changes in distribution, TPPOD exploits an importance-sampling (IS) weight to formulate how updates to the network are weighted. The IS weight ω_i for transition i can be written as

$$\omega_i = \left(\frac{1}{|\mathcal{M}^{replay}|} \cdot \frac{1}{P(i)} \right)^\lambda, \tag{35}$$

where $|\mathcal{M}^{replay}|$ is the size of the replay memory. $\lambda \in [0, 1]$ is the amount of importance-sampling, where $\lambda = 0$ means conducting with no importance-sampling and $\lambda = 1$ represents the full importance-sampling.

Next is to use the collected samples for an update. TPPOD employs the generalized advantage estimator (GAE) [34] to make a compromise between variance and bias. The estimator of the advantage function is calculated by (i.e., Line 11)

$$\begin{aligned} &\hat{A}_{\pi_\theta}(s_t, a_t) = \sum_{i=t}^{T} (\gamma\phi)^{i-t} \delta_{t+i}^s, \\ &\text{where} \quad \delta_t^s = r_t + \gamma V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t), \end{aligned} \tag{36}$$

where ϕ specifies the bias-variance tradeoff, and V_{π_θ} is the value function parameterized by the critic network θ^c.

By using the mini-batch B simpled with prioritization, agent can update the weights of online neural networks by minimizing the loss values, including the critic loss $L(\theta^c)$ and the actor-new loss $L(\theta')$ (Lines 12–13). Specifically, the critic loss $L(\theta^c)$ can be calculated as

$$L(\theta^c) = \mathbb{E}_t \left[\left(V_{\theta'}(s_t) - V_{\theta^c}(s_t) \right)^2 \right]. \tag{37}$$

Afterward, agent implements gradient ascent to update the parameter of the critic network θ^c by minimizing the loss function $L(\theta^c)$ (Line 12). Then agent updates the parameter of the actor-new network θ' by taking advantage of $L(\theta')$ and $\hat{A}_t^s$ (Line 13). The actor-new loss function $L(\theta')$ is given by

$$L(\theta') = \mathbb{E}_t \left[\min \left(\text{rat}_t(\theta) \hat{A}_t^s, \text{clip}\left(\text{rat}_t(\theta), 1-\varepsilon, 1+\varepsilon\right) \hat{A}_t^s \right) \right], \tag{38}$$

where $\varepsilon \in [0, 1]$ is a hyper-parameter. The probability ratio $\text{rat}_t(\theta)$ is used to prevent updates to the policies from becoming excessive, by clipping it to remove the incentive for moving $\text{rat}_t(\theta)$ outside of the interval $[1-\varepsilon, 1+\varepsilon]$ [35]. Thus, $\text{rat}_t(\theta)$ can be computed by the probability ratio of the new policy and the old policy as follows,

$$\text{rat}_t(\theta) = \frac{\pi_{\theta'(a_t|s_t)}}{\pi_{\theta(a_t|s_t)}}. \tag{39}$$

Once the pre-defined step T is completed, agent updates parameters of the actor-old network θ using parameter θ' (i.e., Lines 14–16). Finally, the priority of mini-batch B can be updated by calculating the priority for each transition i with the equation $p_i^\sigma = |\hat{A}_{\pi_\theta, i}| + \epsilon$ (Line 17). The process of network training will last for E episodes to reach the convergence. To conclude, our TPPOD differs from the PPO algorithm in three ways, i.e., demonstration data $\mathcal{D}^s$, replay memory $\mathcal{M}^{replay}$, and the prioritized-experience replay (PER) mechanism.

C. Distributed IoT-Agent Deployment and Consensus

How to use the trained agent to determine the sharding policy for each epoch is an essential question. Instead of centralization, we adopt a distributed agents deployment and consensus scheme, i.e., each identified IoT device maintains a copy of the trained agent. The sharding policy on every epoch, i.e., the number of shards k, partition structure $\mathcal{A}^\Omega$, and the primary selection mode $\mathcal{A}^P$, will be made through a round of consensus by the leader committee L_K, which is comprised of each shard's primary. Next specify the details. First, to start an epoch, a sharding policy proposer $L^p_{\boldsymbol{N}^i}(L^p_{\boldsymbol{N}^i} \in L_K)$ will be selected based on the current epoch randomness. Then, after finishing the period of consensus within its shard, $L^p_{\boldsymbol{N}^i}$ takes the current system state (i.e., the geographical location of IoT devices $\mathbb{G}$, data transmission rate between IoT devices and their linked base stations $\boldsymbol{\Upsilon}$, and the available computing power $\mathbb{F}$ for each IoT device), as the input of its agent, thus making a sharding policy $\mathcal{A}_t$. Finally, the proposed $\mathcal{A}_t$ needs to be agreed by L_K via four phases: 1) *Broadcast*, $L^p_{\boldsymbol{N}^i}$ sends the system state in its view $\mathcal{S}_t(L^p_{\boldsymbol{N}^i})$, $\mathcal{A}_t$, and the parameter of its neural networks to other leaders in L_K for validation; 2) *Echo*, these leaders who received the messages need to broadcast it with an echo tag. This echo tag is used to indicate the difference between $\mathcal{S}_t(L^p_{\boldsymbol{N}^i})$ and the system state from the view of a verifier leader. An error threshold Δs is predefined to justify if the difference is reasonable, and $\mathcal{A}_t$ with a different $\leqslant \Delta s$ can be acceptable and the tag is then set to be y; 3) *Accept*, once an honest leader has received more than $\frac{1}{2}$ fractions of leader messages with the tag y, it can accept $\mathcal{A}_t$ as its sharding policy and broadcast $\mathcal{A}_t$ to other leaders together with a tag $accept$; 4) *Update*, if $L^p_{\boldsymbol{N}^i}$ receives more than the half of messages with tag $accept$, it will take a batch of transitions $[\mathcal{A}_t, \mathcal{S}_t(L^p_{\boldsymbol{N}^i}), \mathcal{R}_t]$ from its replay memory $\mathcal{M}^{replay}$ and broadcast them together with the neural network

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION PARAMETERS

Parameter	Value
Number of IoT devices, N	[20, 90, 100, 110, 200]
Number of base stations, M	[5, 9, 10, 11, 20]
Average transaction size, $\mathcal{S}^T$	64 B
Block size, $\mathcal{S}^B$	8 MB
Number of rounds for consensus at one epoch, r	1000
The transmit power, h_n	[0-24] dBm
The background noise variance, δ^2	100 dBm
The channel bandwidth, B	20 MHz
The data transmission rate of two base stations	2 Gbps
Cost of CPU cycles for signature validation, α	2 MHz
Cost of CPU cycles for MAC generation & validation, β	1 MHz
Batch size, $\mathcal{M}$	3
The available computing power that cyber attacker employs to attack IoT nodes, ξ	5 GHz
Maximum allowable transmission delay, $\mathcal{T}$	100 s

parameters to other leaders. To achieve a consistent update, each leader needs to broadcast received messages to the IoT devices within its local shard; thus, all agents can keep consistency via updates.

VI. Numerical Results

A. Simulation Setting

We conduct the shard-driven simulations of SmartChain with N IoT devices and M base stations scattering over a geographical area of size 1 km $\times$ 1 km. The consensus protocols adopted by shards are the PBFT, which sets the shard Byzantine fault tolerance to $1/3$ and the system Byzantine fault tolerance equal to $1/4$. For each epoch, IoT devices are initially positioned at random locations within the defined geographical area and then travel at random speeds for all time slots, whereas base stations are fixed to the random positions during an epoch. Inspired by [10], [19], [23], [25], the parameter settings in our simulation are configured in Table I. The channel gain g_n is computed by using the distance-dependent path-loss model $L(d_{mn}) = 15.3 + 37.6 \log_{10} d_{mn}$. For cyberattack, we assume the initial capabilities of IoT device's computing defense, as well as the base stations, follow the Poisson distribution with parameter $\lambda_{Cn} = 20$ GHz and $\lambda_{Cm} = 200$ GHz, respectively.

Besides, we use PyTorch 1.1.0 to implement the DNN and each distributed agent has the same DNN structure. The actor-network and critic-network have two and three hidden layers, respectively, in which each layer with $(N + M) \times 3$ neurons. The activation functions of hidden layers are TanH. We use Adam optimizer to optimize the values of parameters in DNN models with a learning rate of 0.003 and update the DNN once per 2048 steps. The clip parameter ε is 0.2. The demonstration dataset size $|\mathcal{D}^s|$ and replay memory size $|\mathcal{M}^{replay}|$ are set to 100 and 20,000, respectively. The batch size for the DRL training is 64. In particular, the priority parameter σ and importance-sampling parameter λ are 0.4 and 0.6, respectively. After each sampling, λ is set to increase by 0.001, from the initial 0.6 to 1.0. Lastly, each distributed DRL agent applies a discount factor of 0.98.

The implementation of several sharding strategies for comparison are explained as follows:

- *Sharding_PPO* makes sharding strategy by using PPO [35], which is a state-of-the-art algorithm with excellent stability and reliability.
- *DQNSB_* [19] provides a DQN-based sharding strategy, which does not consider the dynamics of IoT devices as well as shards reconfiguration.
- *Sharding_Static* is a baseline adopted by most previous sharding systems [4], [5], [6], [7], which sets a fixed shard number, fixed partition structure, and the fixed primary selection mode.
- *Sharding_Random* is to randomly select sharding actions for each distributed agent at each time slot.

B. Evaluation of Training Performance

First of all, we examine the performance of two crucial hyper-parameters in Fig. 5, i.e., learning rate lr and mini-batch size bs. We observe that trends of the three lines in each subgraph roughly keep the same, blue line $bs = 64$ remains the most steady, while the fluctuation of black line $bs = 16$ is the heaviest. This demonstrates that a bigger mini-batch size can enhance the stability of training process, making the value of loss function converge to a relatively more stable mini-range. Compared to lines with the same color, we find that the training process for learning rate 0.003 yields the best performance, i.e., the convergence reaches the fastest and the curves show most stable after convergence. By contrast, when $lr = 0.0003$, the training has the slowest learning speed. Therefore, we set $lr = 0.003$ and $bs = 64$ in our subsequent experiments.

We then evaluate the training performance in terms of average system reward and throughput. Here, we set a simulation environment with 100 IoT devices, 10 base stations, and 25 maximum shard number for each time slot. We test with the fixed shard number (i.e., $k = 17$), fixed primary selection mode_1 ($a^k = 0$), and fixed primary selection mode_2 ($a^k = 1$), respectively. Fig. 6(a) shows that SmartChain converges quickly and yields the best performance on average system reward. Besides, we notice that both shard number and the primary selection mode can affect average system reward, but the two primary-selection modes show little difference in performance.

Fig. 6(b) illustrates the throughput. To compare with DQNSB_ [19], we make a similar setting of 200 IoT devices, 20 base stations, and 50 maximum allowable shard number. The ratio of malicious nodes for DQNSB_ [19] is set to 0%. The fixed shard number and primary selection mode for Sharding_Static are 28 and 0, respectively. It is clear that SmartChain yields the best performance. On one hand, SmartChain converges faster than Sharding_PPO, where it can converge from the 200th episode while Sharding_PPO achieves stability until the episodes are over 500. This is because the introduced TL mechanism improves the learning capacity. As for DQNSB_ [19], it shows the lowest convergence rate due to the large action space of sharding system, whereas other PPO-like strategies can avoid long-time training by setting a proper learning rate. On the other hand, SmartChain reaches the highest TPS that is almost the same as Sharding_PPO, outperforming the other three. The reason is Sharding_Static and Sharding_Random

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

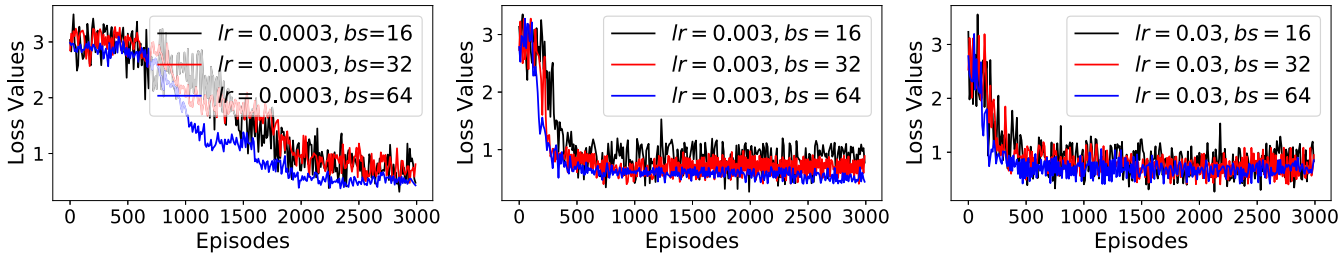


Fig. 5. Curves of loss functions when N is fixed at 20 and M is fixed at 5, under different candidate values respectively for the learning rate and mini-batch size: $lr = 0.0003/0.003/0.03$; $bs = 16/32/64$, with different combination settings.

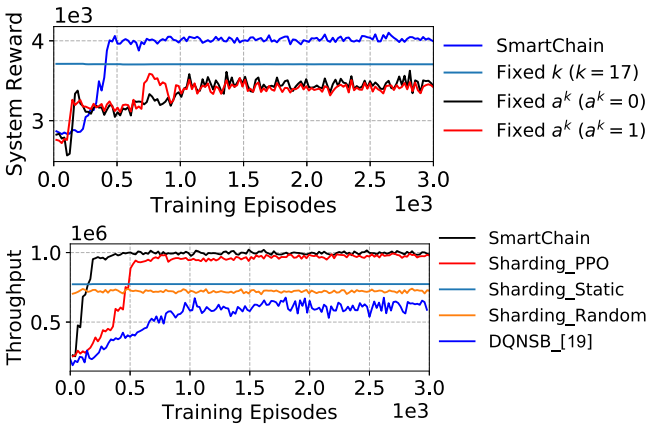


Fig. 6. Evaluations of training performance, i.e., (a) Comparison of the average system reward with fixed actions when $N = 100$, (b) Comparison of the throughput with different sharding strategies when $N = 200$.

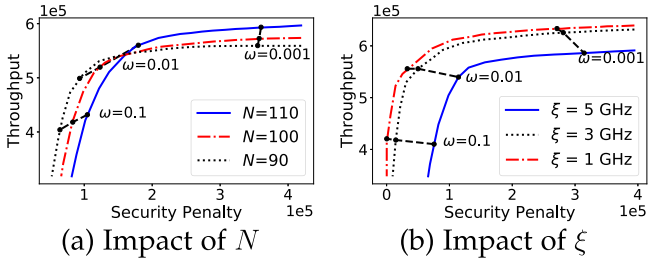


Fig. 7. Pareto-optimal points of shards partitioning.

TABLE II
COMPUTATIONAL PERFORMANCE OF THE SHARDING METHODS

Method	DQNSB _ [19]	Sharding _PPO	SmartChain
Average training time per episode (s)	13.86	8.58	14.04
Number of episodes	1000	600	400
Total training time (h)	3.85	1.43	1.56
Average test time (ms)	5.45	4.91	4.15

cannot deal with a dynamic sharding environment. DQNSB_ [19] has the lowest TPS because it was designed to conduct sharding optimization with bounded shards (e.g., the security constraint based on different network trust levels).

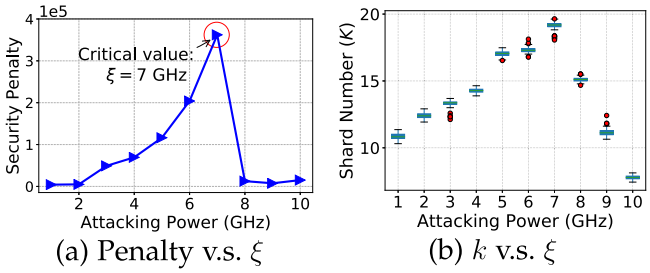
Table II summarizes the computational performance in terms of training and execution. SmartChain runs the highest average training time per episode since it combines demonstration data and self-generated data for the replay training. However, SmartChain requires the fewest number of episodes to reach convergence because the demonstration data can be utilized to improve the actor_new's efficiency on action selection. The total training time is shortest in Sharding_PPO because of sample efficiency, longer in SmartChain because of the introduced combination replay, and longest in DQNSB_ [19] because of the large Q-value computing. For execution, it can be observed that SmartChain exhibits the smallest average test time in the order of milliseconds, implying that our method with a trained agent can be adapt to the complex dynamic IoT environments.

C. Pareto Optimal Trade-Off

Next, we illustrate the trade-off between the conflicting goal of maximizing throughput and security by using the Pareto curve. We first test the tradeoff under varying scales of nodes, as shown in Fig. 7(a). The parameter settings are [90, 9, 22], [100, 10, 25], and [110, 11, 26], each of which represents the number for IoT devices N, base stations M, and the allowable maximum shards $\hat{K}$, respectively. Then, we examine the impact of attacking power ξ in Fig. 7(b).

The Pareto curve shows that the trade-off between the goals of maximizing the throughput $\sum_{k=1}^{K} \mathbb{T}_k(\boldsymbol{N}_k)$ and minimizing the security penalty $\sum_{k=1}^{K} \mathcal{P}_k(\boldsymbol{N}_k)$, in which we can increase TPS yet at the expenses of increasing the security risks (i.e., a larger security penalty). The results also demonstrate the self-adaptive capacity of SmartChain, which can optimize the average system reward by flexibly adapting to different performance requirements on throughput and security. We explain it as follows: 1) $\omega < 0.001$, designed for the solely TPS maximization; 2) $0.001 \leq \omega < 0.01$, designed for the prioritized TPS maximization; 3) $0.01 \leq \omega < 0.1$, designed for balancing TPS maximization and security penalty minimization; and 4) $0.1 \leq \omega \leq 1$, designed for the prioritized security penalty minimization. In general, the first two regions are suitable for the consortium blockchain with high security guarantees, while the last two regions are more applicable to public blockchain.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.



(a) Penalty v.s. ξ (b) k v.s. ξ

Fig. 8. Evaluation of security performance.

D. Evaluation of Security Performance

In this group of simulations, under the attacking power ξ ranging from 1 GHz to 10 GHz, we investigate the security performance as illustrated in Fig. 8. Both security penalty and shard number are calculated by averaging 3,000 episodes. From Fig. 8(a), we observe that security penalty is increasing when ξ increases from 1 GHz to 7 GHz. However, from $\xi = 7$ GHz, the penalty reduces to a relatively small value. This suggests that the security penalty of sharding system cannot be enlarged without a limit. Particularly, when $\xi \geq 8$ GHz, SmartChain becomes unsafe because the shard Byzantine fault tolerance is set to 1/3 and the system Byzantine fault tolerance is 1/4. Fig. 8(b) shows the average number of shards corresponding to each episode, which displays the same trends as security penalty. This can explain why security penalty will not increase indefinitely. To maintain a safe system, SmartChain can decrease security penalty when the penalty gets too large by adjusting the shard number k.

E. Evaluation of TPS Performance

To evaluate the throughput, we explore the impact of different parameters, including block size ($\mathcal{S}^B$), average transaction size ($\mathcal{S}^T$), and consensus round (r). Fig. 9(a) indicates the throughput with an increasing number of $\mathcal{S}^B$, from 1 MB to 10 MB. We find that the TPS of SmartChain, Sharding_PPO and DQNSB_ [19] increase significantly, Sharding_Static increases slightly, and Sharding_Random fluctuates within a small range. This implies that a sharding system can handle more transactions when increasing the block size. We also observe that the TPS will not become infinitely large as $\mathcal{S}^B$ increases, since a larger $\mathcal{S}^B$ can also increase consensus latency and reconfiguration latency, thereby decreasing the TPS.

Fig. 9(b) shows the result of throughput while average transaction size $\mathcal{S}^T$ is ranging from 32 KB to 512 KB. As expected, the TPS for all decrease as $\mathcal{S}^T$ increases. The reason is obvious that one block contains fewer transactions due to a larger transaction size. We see that SmartChain consistently outperforms the baseline with highest TPS varying with different $\mathcal{S}^T$ values. Fig. 9(c) examines the throughput under different consensus rounds r. We set r for each epoch ranging from 200 to 1000. Obviously, the TPS of three strategies (i.e., SmartChain, Sharding_PPO, Sharding_Static) can increase as r increases, while Sharding_Random fluctuates up and down. The reason is raising r means that shard will have more time to process transactions within a certain time period, which increases the system's transactional throughput.

F. Evaluation of PER Mechanism

The prioritized-experience replay (PER) mechanism has significant impacts on training. We evaluate SmartChain with and without the PER by adjusting two crucial parameters, i.e., σ (see (34)) and λ (see (35)).

We show the impact of (34) with different σ values in Fig. 10(a), compared to the uniform sampling case (i.e., $\sigma = 0$, dashed horizontal line). We see that the sample probabilities of each transition in the uniform baseline are the same equaling to 5×10^{-5}. This is because the priorities of all transitions are equal to 1 when $\sigma = 0$, for which there is a total of 20,000 transitions in the sum-tree data structure [33]. We also observe that the sample probability is increasing as the transition advantage increases when $\sigma \neq 0$. Notice that $\sigma = 0.4$ shows the most uniform distribution on the sample probability. Hence, we have applied $\sigma = 0.4$ in our simulations.

Next, we evaluate the impact of (35) varying with λ, compared to the uniform baseline (i.e., $\sigma = 0$, $\lambda = 0$) in Fig. 10(b). The figure has three cases: the full importance-sampling case (i.e., $\lambda = 1$), no importance-sampling case (i.e., $\lambda = 0$), and our chosen parameters (i.e., $\lambda = 0.6 \rightarrow 1$). We observe that leveraging importance-sampling improves the training speed and the average system reward eventually converges to a higher value. No importance-sampling case has the same impact on average system reward as the uniform sampling. Further, we find that linearly annealing λ from the initial value 0.6 to 1 yields the best performance. Thus, we choose $\sigma = 0.4$ and $\lambda = 0.6 \rightarrow 1$ in practice.

Finally, we consider the impact of the amount of demonstration data $|\mathcal{D}^s|$ on the average system reward, as shown in Fig. 10(c). The demonstration dataset in our TPPOD with uniform sampling is the size of 100. We find that TPPOD acquire a faster convergence rate and a higher score than that of the Uniform baseline, thanks to the PER. Moreover, we see that TPPOD_PER_100 achieves a higher score than TPPOD_PER_300, TPPOD_PER_800, and TPPOD_PER_2000; their scores are higher when $|\mathcal{D}^s|$ is smaller. This proves that SmartChain can maximize the system reward by only leveraging small amounts of demonstration data due to the few-shot learning in transfer learning [11].

When $|\mathcal{D}^s| = 2{,}000$, TPPOD_PER_2000 exhibits the lowest score. This is because a large $|\mathcal{D}^s|$ inevitably leads to a relatively small set of self-generated data in the replay memory. However, for the actor_network in TPPOD, the output action distribution has constraints on its entropy, i.e., a larger entropy is preferred. This requires adequate self-generated data to ensure sufficient exploration by interacting with the target environments. This implies that SmartChain utilizes small sets of demonstration data in practice to largely improve the training performance by combing the PER mechanism; thus, it can scale well under the time-varying environment in IoT blockchain.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

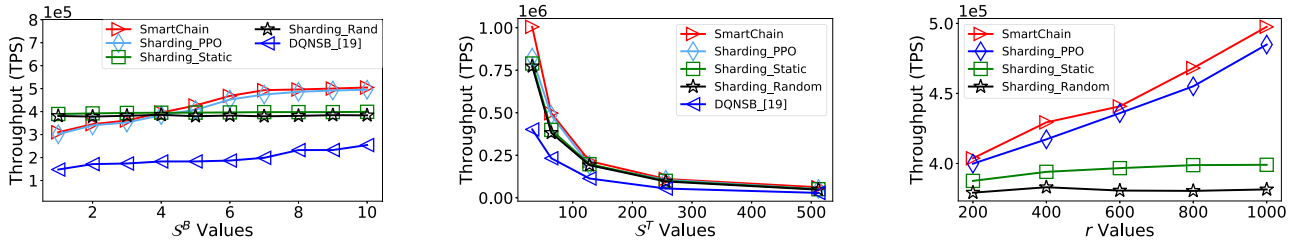


Fig. 9. TPS performance under different sharding strategies, while fixing $N = 100$, $M = 10$ and $\hat{K} = 25$. The parameters $\mathcal{S}^B$, $\mathcal{S}^T$ and r stand for block size, average transaction size, and consensus round, respectively.

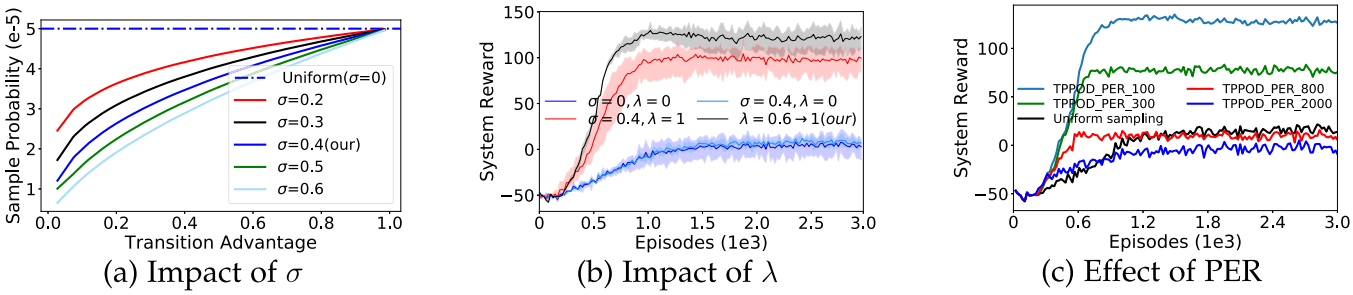


Fig. 10. (a) Effect of priority σ on (34), while varying σ within $\{0, 0.2, 0.3, 0.4, 0.5, 0.6\}$, and fixing $N = 20$, $M = 5$, $K = 4$. (b) Effect of importance-sampling λ on (35), under different λ within $\{0, 1, 0.6\}$, while fixing $N = 20$, $M = 5$, $K = 4$. (c) Effect of demonstration data size $|\mathcal{D}^s|$ on sharding performance, while fixing the replay memory size $|\mathcal{M}^{replay}| = 20{,}000$, and varying $|\mathcal{D}^s|$ within $\{100, 300, 800, 2{,}000\}$, under $N = 20$, $M = 5$ and $\hat{K} = 4$.

VII. Conclusion

In this paper, we have proposed SmartChain to maximize the throughput while minimizing the security penalty for dynamic and heterogeneous sharded IoT blockchains. To evaluate performance, we provide a quantitative measurement of consensus latency, reconfiguration latency, and the penalty for violation of sharding security. To enhance learning performance, we present an algorithm, Transferable PPO with Demonstrations (TPPOD), which exploits small amounts of demonstration data from successful experiences to massively accelerate the training process when partitioning and reconfiguring shards. SmartChain implements a dynamic and adaptive sharding strategy with a long-term tradeoff for higher TPS and lower security penalty. SmartChain delivers a decrease in training time of 60% and 80% compared with DQNSB_ [19] and Sharding_PPO, respectively. SmartChain reaches a new state-of-the-art, outperforming the PPO-based sharding with uniform sampling in terms of 8.3 times the average system reward.

In future work, we plan to further focus on the features of real-world IoT tasks and improve the applicability of SmartChain to the real IoT systems. Some general and other specific IoT setups considering beyond 5G/6G IoT networks with advanced technologies such as millimeter wave, terahertz waves, and reconfigurable intelligent surfaces will be applied. In addition, the base station may be offline in real complex IoT scenarios, which may cause the missing of state blocks for some shards. Thus, enhancing system resilience deserves further research.

References

[1] Z. Xiong, Y. Zhang, D. Niyato, P. Wang, and Z. Han, "When mobile blockchain meets edge computing," *IEEE Commun. Mag.*, vol. 56, no. 8, pp. 33–39, Aug. 2018.

[2] P. Castagno, V. Mancuso, M. Sereno, and M. A. Marsan, "A simple model of MTC flows applied to smart factories," *IEEE Trans. Mobile Comput.*, vol. 20, no. 10, pp. 2906–2923, Oct. 2021.

[3] G. Wang, Z. J. Shi, M. Nixon, and S. Han, "SoK: Sharding on blockchain," in *Proc. 1st ACM Conf. Adv. Financial Technol.*, 2019, pp. 41–61.

[4] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 17–30.

[5] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 931–948.

[6] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *Proc. IEEE Symp. Secur. Privacy*, 2018, pp. 583–598.

[7] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *Proc. 16th {USENIX} Symp. Netw.ed Syst. Des. Implementation*, 2019, pp. 95–112.

[8] L. N. Nguyen, T. D. Nguyen, T. N. Dinh, and M. T. Thai, "OptChain: Optimal transactions placement for scalable blockchain sharding," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 525–535.

[9] S. Li, M. Yu, C.-S. Yang, A. S. Avestimehr, S. Kannan, and P. Viswanath, "PolyShard: Coded sharding achieves linearly scaling efficiency and security simultaneously," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 249–261, 2021.

[10] J. Zhang, Z. Hong, X. Qiu, Y. Zhan, S. Guo, and W. Chen, "SkyChain: A deep reinforcement learning-empowered dynamic blockchain sharding system," in *Proc. 49th Int. Conf. Parallel Process.*, 2020, pp. 1–11.

[11] Z. Zhu, K. Lin, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," 2020, *arXiv:2009.07888*.

[12] W. Chen, X. Qiu, T. Cai, H.-N. Dai, Z. Zheng, and Y. Zhang, "Deep reinforcement learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 23, no. 3, pp. 1659–1692, Third Quarter 2021.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.

[13] S. Misra, A. Mukherjee, A. Roy, N. Saurabh, Y. Rahulamathavan, and M. Rajarajan, "Blockchain at the edge: Performance of resource-constrained IoT networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 174–183, Jan. 2021.
[14] Y. Feng, W. Zhang, X. Luo, and B. Zhang, "A consortium blockchain-based access control framework with dynamic orderer node selection for 5G-enabled industrial IoT," *IEEE Trans. Ind. Inform.*, vol. 18, no. 4, pp. 2840–2848, Apr. 2022.
[15] Y. Liu, K. Wang, K. Qian, M. Du, and S. Guo, "Tornado: Enabling blockchain in heterogeneous internet of things through a space-structured approach," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1273–1286, Feb. 2020.
[16] P. Zheng et al., "Aeolus: Distributed execution of permissioned blockchain transactions via state sharding," *IEEE Trans. Ind. Inform.*, vol. 18, no. 12, pp. 9227–9238, Dec. 2022.
[17] L. Liu et al., "BS-IoT: Blockchain based software defined network framework for Internet of Things," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2020, pp. 496–501.
[18] L. Xue, W. Yang, W. Chen, and L. Huang, "STBC: A novel blockchain-based spectrum trading solution," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 1, pp. 13–30, Mar. 2022.
[19] J. Yun, Y. Goh, and J.-M. Chung, "DQN-based optimization framework for secure sharded blockchain systems," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 708–722, Jan. 2021.
[20] S. Yuan et al., "Sharding for blockchain based mobile edge computing system: A deep reinforcement learning approach," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.
[21] Z. Yang, R. Yang, F. R. Yu, M. Li, Y. Zhang, and Y. Teng, "Sharded blockchain for collaborative computing in the internet of things: Combined of dynamic clustering and deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16 494–16 509, Sep. 2022.
[22] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Trans. Comput. Syst.*, vol. 20, no. 4, pp. 398–461, 2002.
[23] M. Liu, F. R. Yu, Y. Teng, V. C. Leung, and M. Song, "Performance optimization for blockchain-enabled industrial Internet of Things (IIoT) systems: A deep reinforcement learning approach," *IEEE Trans. Ind. Inform.*, vol. 15, no. 6, pp. 3559–3570, Jun. 2019.
[24] T. S. Rappaport et al., *Wireless Communications: Principles and Practice*. Englewood Cliffs, NJ, USA: Prentice Hall, 1996, vol. 2.
[25] Q.-V. Pham, H. T. Nguyen, Z. Han, and W.-J. Hwang, "Coalitional games for computation offloading in NOMA-enabled multi-access edge computing," *IEEE Trans. Veh. Technol*, vol. 69, no. 2, pp. 1982–1993, Feb. 2020.
[26] A. Garnaev, M. Baykal-Gursoy, and H. V. Poor, "Security games with unknown adversarial strategies," *IEEE Trans. Cybern.*, vol. 46, no. 10, pp. 2291–2299, Oct. 2016.
[27] S. Feng, Z. Xiong, D. Niyato, and P. Wang, "Dynamic resource management to defend against advanced persistent threats in fog computing: A game theoretic approach," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 995–1007, Third Quarter 2021.
[28] C. Qiu, H. Yao, F. R. Yu, C. Jiang, and S. Guo, "A service-oriented permissioned blockchain for the Internet of Things," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 203–215, Mar./Apr. 2020.
[29] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proc. IEEE 40th Annu. Symp. Found. Comput. Sci.*, 1999, pp. 120–130.
[30] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch, "Verifiable delay functions," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 2018, pp. 757–788.
[31] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proc. Int. Conf. Manage. Data*, 2019, pp. 123–140.
[32] X. Cai et al., "A sharding scheme based many-objective optimization algorithm for enhancing security in blockchain-enabled industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 17, no. 11, pp. 7650–7658, Nov. 2021.
[33] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2015, *arXiv:1511.05952*.
[34] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.



Ting Cai (Member, IEEE) received the PhD degree from Sun Yat-Sen University, Guangzhou, China, in 2022. She is currently an associate professor with the Department of Computer Science with Hubei University of Technology, Wuhan, China. Her research interests are in the area of blockchain, deep reinforcement learning, edge/cloud computing, and federated learning.



Wuhui Chen (Member, IEEE) received the bachelor's degree from Northeast University, Shengyang, China, in 2008, and the master's and PhD degrees from the University of Aizu, Aizuwakamatsu, Japan, in 2011 and 2014, respectively. From 2014 to 2016, he was a research fellow with the Japan Society for the Promotion of Science, Tokyo, Japan. From 2016 to 2017, he was a researcher with the University of Aizu. He is currently an Associate Professor with Sun Yat-Sen University, Guangzhou, China. His current research interests include edge/cloud computing, cloud robotics, and blockchain.



Jianting Zhang received the bachelor's and master's degrees from Sun Yat-Sen University, Guangzhou, China, in 2019 and 2022, respectively. He is currently working toward the PhD degree with the Department of Computer Science at Purdue University, West Lafayette, USA. His research interests include the field of blockchain, smart contracts, and consensus algorithms.



Zibin Zheng (Fellow, IEEE) received the PhD degree from the Chinese University of Hong Kong, in 2011. He is currently a professor with the School of Data and Computer Science with Sun Yat-Sen University, China. He serves as chairman of the Software Engineering Department. He published more than 120 international journal and conference papers, including 3 ESI highly cited papers. According to Google Scholar, his papers have more than 7000 citations, with an H-index of 42. His research interests include blockchain, services computing, software engineering, and financial Big Data. He was a recipient of several awards, including the Top 50 Influential Papers in Blockchain of 2018, the ACM SIGSOFT Distinguished Paper Award at ICSE2010, the Best Student Paper Award at ICWS2010. He served as BlockSys'19 and CollaborateCom'16 General Co-Chair, SC2'19, ICIOT'18 and IoV'14 PC Co-Chair.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:21:00 UTC from IEEE Xplore. Restrictions apply.